

Parallel Sorting

Ana Costa Pereira, Josh Davies

26/06/2026

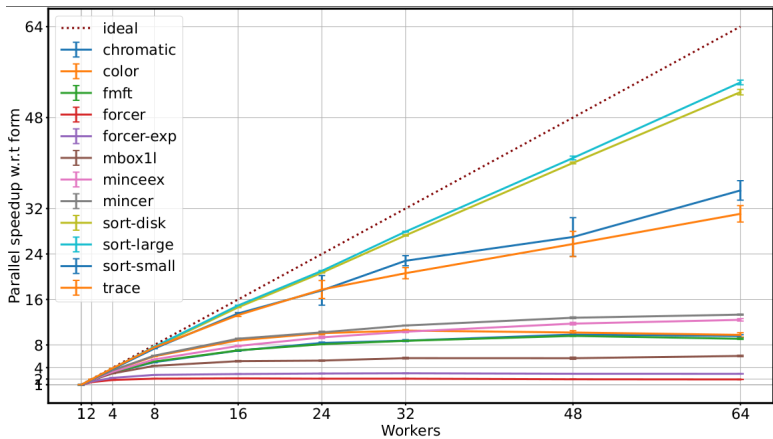


- ① Parallel scaling at higher thread count
- ② SortBot Merging
- ③ SortBot Bottlenecks and changes
- ④ Changes in the SortBot Merge
- ⑤ SortBot Locking

Outline of talk

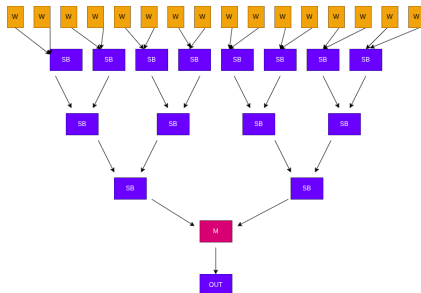
Parallel scalings: study of scaling at higher thread number of tform;

- Overview of sorting process in parallel at the [SortBot](#) level;
- Benchmarks performance at different thread count;
- Changes implemented at the level of [SortBlocks](#) and new benchmark results;



SortBot Merging

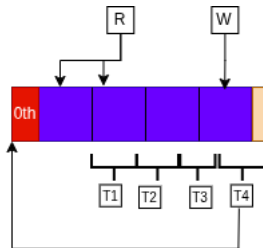
- **Master** $\rightarrow$ distributes terms to the workers;
- **Worker** $\rightarrow$ independent stream of sorted terms;
- N **workers** $\rightarrow N - 2$ **SortBlocks** are created;
- **SortBlocks** routine starts;



- **Master** performs the final merge and writes output.

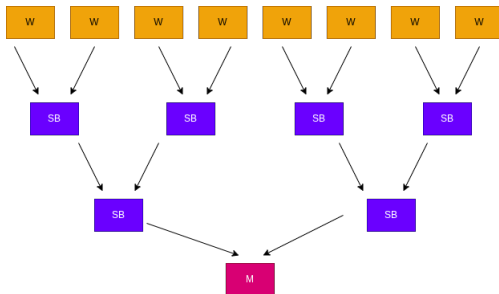
SortBot: Overlapping of terms

- SortBot $\rightarrow$ blocks;

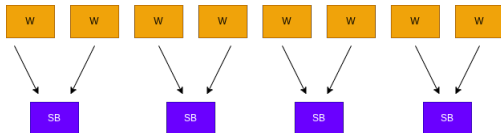


- Terms are allowed to overlap between blocks;
- 0th block** - the overlap in the end of term is copied to **0th block** so that the terms are contiguous.

- **Bottlenecks:** reading of the input with **division of terms of workers** to **SortBlocks**; inverse tree at the end with the **writing to the file by the Master**;

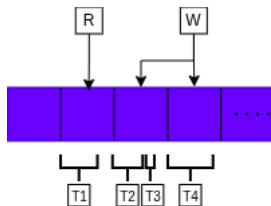


Bottlenecks

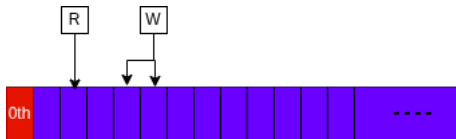


- Default buffer size increased over time → bigger **SortBlocks**;
- Result from **worker** fits into **single block** → SortBotTree mechanism does not parallelize
- If a **term** does not fit at the end of a block: Writer fills the **SortBlocks** all the way up to the end of the block, **overlapping** to the next;
- Once a block is completely full, it reaches the top and moves on to the next block.

Change: SortBot Block size

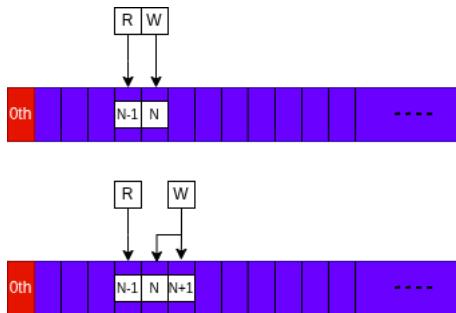


- Require fewer SortBlocks - master buffers not expanded so often;
- Whole terms must fit into the block: if it does not fit, it goes into the next block;



- Writing thread always locks pair of locks - the current thread and the previous thread;
- Reading threads locks the block where it is reading from;

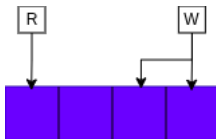
Change: SortBot Locking



- Writer (W) detects if there is a Reader (R) waiting for it;
- W filling the blocks tries to obtain a lock of the block before the current fill block;
- If W does not obtain the lock, R thread already has the lock - very soon will be done and ready for more terms;
- The W moves to the next block;

- The number of blocks in each SortBot has to be at least 4 (due to the locking mechanism) - which corresponds to a minimum of 8 for **NUMBEROFBLOCKSINSORT**;
- There is a minimum number of terms that have to fit the block: **MINNUMBEROFTERMS**;
- Make sure a minimum number of terms is written: never make empty block - **MINWRITENNUMBEROFTERMS**;

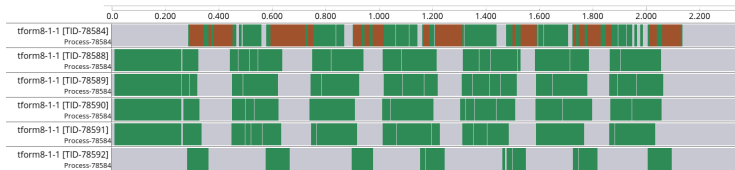
- $\text{NUMBEROFBLOCKSINSORT} = 10 \rightarrow 8$
- $\text{MINNUMBEROFTERMS} = 10 \rightarrow 1$
- $\text{MINWRITENUMBEROFTERMS} = 1$



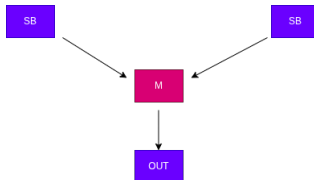
- No decrease in performance for any tests;
- Performance improvement for benchmarks where term merging is expensive (PolyRatFun, PolyRatFunExpand)
- Forcer, mbox1l

Results after changes: Profiler

- Test:



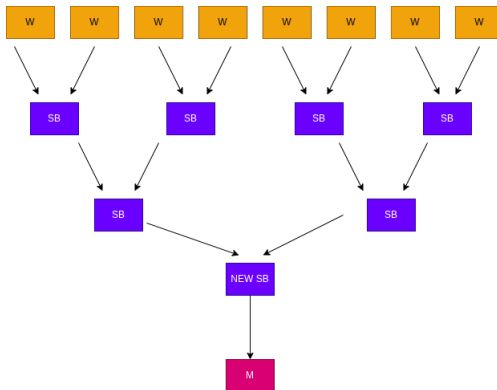
Bottlenecks



- End of the tree: **Master** performs last sorting as well as writing of terms into the output;
- Performance could be increased by having one last stage of sorting of the last 2 **SortBlocks** and **Master** only writing;

Addition of new SortBot

- Add a new SortBot at the end of the merging tree to do the sort of the streams that comes from last 2 SortBlocks;
- Master only does the writing.



- Parallel performance at higher thread needs to be improved;
- [SortBot](#) changes - 20% improvement for some tests;
- Current working on improving [SortBot](#) parallelization by addition of extra SortBot at the end of the tree.